

UPPSALA UNIVERSITY



BACHELOR'S THESIS

Martingale Matching

Authors:

Anton VON
NANDELSTADH

June 16, 2026

Contents

1	Introduction	2
2	Background	2
2.1	SDE Preliminaries	2
2.2	Generative Models	8
3	Algorithm	11
3.1	Training	11
3.2	Generation	18
4	Results	19
5	Conclusion	24

1 Introduction

This thesis aims to develop a novel method for learning the stochastic differential equation of a diffusion-based generative model, which we call martingale matching. We present this as an alternative to the predominant frameworks of score matching [13] and flow matching [8].

The first diffusion models were discrete-time Gaussian processes [11, 2]. These were later reformulated as stochastic differential equations (SDEs) [13]. In a diffusion model, the probability path interpolating between a simple distribution and the data distribution is created through a forward noising process modeled by an SDE. Inspired by the reversal of the forward noising process [1], these SDEs are chosen such that they have marginal probabilities which can parametrise the generator of the diffusion process using the score function, hence the name score matching [12, 9]. Our discussions regarding the framework of flow/score matching mostly follow the conventions used by [9, 3]. Specifically, we consider the SDE we want to learn as moving forward in time, unlike the theory of time-reversal used in the older score matching literature, such as [13].

Our novel approach is to learn the generator of the diffusion process directly, instead of using the score to do this implicitly. We do this by enforcing a known martingale from the basic theory of SDEs. We then use deviations from the martingale to construct a new loss function, based on which we train and learn the drift and diffusion functions. We present our results applying this new methodology to various toy datasets, and discuss the possibility of using it to train on high-dimensional data. In order to do this, we first develop some of the theory of stochastic differential equations, and the general theory of diffusion-based generative models.

2 Background

2.1 SDE Preliminaries

In order to understand denoising diffusion models, we first need to go through some preliminaries regarding stochastic processes and stochastic differential equations. In what follows, we assume basic knowledge of measure theory and measure-theoretic probability theory. We begin by expanding our regular probability space, defined by a triple, into a filtered probability space, defined

by a quadruple.

Definition 2.1 (Filtration)

Let $(\Omega, \mathcal{F}, P)$ be a probability space and let T be an ordered index set. For each $t \in T$, let $\mathcal{F}_t$ be a sub- σ -algebra of $\mathcal{F}$. Then $\{\mathcal{F}_t\}_{t \in T}$ is a filtration if $\mathcal{F}_k \subset \mathcal{F}_l$ for each $k \leq l$. Then $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, P)$ is called a filtered probability space.

We are now ready for one of the central definitions of modern probability theory, that of a martingale.

Definition 2.2 (Martingale)

A stochastic process $X : T \times \Omega \rightarrow \mathbb{R}$ on a probability space $(\Omega, \mathcal{F}, P)$ is a martingale w.r.t. the filtration $\{\mathcal{F}_t\}$ (of $(\Omega, \mathcal{F}, P)$) if

1. X is adapted to the filtration. That is, X_t is $\mathcal{F}_t$ measurable for each $t \in T$.
2. For each t , $\mathbb{E}[|X_t|] < \infty$
3. For all s and t with $s < t$, $\mathbb{E}[X_t | \mathcal{F}_s] = X_s$.

A very important example of a martingale is that of Brownian motion.

Definition 2.3 (Brownian motion)

The standard d -dimensional Brownian motion is a stochastic process $\{B_t\}$, $t \in [0, T]$, such that

- $B_0 = 0$
- for all k , and all $0 = t_0 < t_1 < \dots < t_k = T$, the random variables

$$B_{t_k} - B_{t_{k-1}} \sim N(0, (t_k - t_{k-1})I)$$

are independent, where I is the d -dimensional identity matrix.

- B_t is almost surely continuous in t

We now present the Itô integral, which is essential for our definition of stochastic differential equations. For the definition to be well-defined, we need to restrict the class of functions ν to which the definition applies.

Definition 2.4

Let $\nu = \nu(S, T)$ be the class of functions

$$f(t, \omega) : [0, \infty) \times \Omega \rightarrow \mathbb{R}$$

such that

1. $(t, \omega) \rightarrow f(t, \omega)$ is $\mathcal{B} \times \mathcal{F}$ measurable, where $\mathcal{B}$ denotes the Borel σ -algebra on $[0, \infty)$.
2. $f(t, \cdot)$ is $\mathcal{F}_t$ measurable for every t .
3. $\mathbb{E}[\int_S^T f(t, \omega)^2 dt] < \infty$.

where $[S, T]$ is a time interval.

Similarly to how the Lebesgue integral is constructed from simple functions, one can define a class of elementary functions, and then the integral of $f \in \nu$ can be defined as an L^2 limit of these elementary functions.

Definition 2.5 (Elementary function)

A function $\phi \in \nu$ is called elementary if it has the form

$$\phi(t, \omega) = \sum_j e_j(\omega) \chi_{[t_j, t_{j+1})}(t). \quad (1)$$

where the sum is taken over some finite partition and each e_j is $\mathcal{F}_{t_j}$ measurable.

Now, again similarly to how we do Lebesgue integration, we begin by defining the integral of the elementary functions with respect to Brownian motion.

Definition 2.6 (Integral of an elementary function)

For elementary function $\phi(t, \omega)$, the integral is defined according to

$$\int_S^T \phi(t, \omega) dB_t(\omega) = \sum_{j \geq 0} e_j(\omega) [B_{t_{j+1}} - B_{t_j}](\omega). \quad (2)$$

One can then show that for $f \in \nu$, one can choose elementary functions $\phi_n \in \nu$ converging to f in L^2 norm. For a detailed construction of the Itô integral, see [10] or [6]. We get the definition below.

Definition 2.7 (Itô integral)

Let $f \in \nu(S, T)$. Then the Itô integral of f (from S to T) is defined by

$$\int_S^T f(t, \omega) dB_t(\omega) = \lim_{n \rightarrow \infty} \int_S^T \phi_n(t, \omega) dB_t(\omega) \quad (3)$$

where $\{\phi_n\}$ is a sequence of elementary functions such that

$$\mathbb{E} \left[\int_S^T (f(t, \omega) - \phi_n(t, \omega))^2 dt \right] \rightarrow 0 \quad \text{as } n \rightarrow \infty \quad (4)$$

Note that by the construction [10], the Itô integral of $f \in \nu$ is independent of which sequence of elementary functions we choose. The definition of the Itô integral can moreover be naturally extended to integration w.r.t. d -dimensional Brownian motion, see e.g. [10] or [6]. An important corollary of the construction of the Itô integral is the Itô isometry.

Corollary 2.1 (Itô isometry)

For $f \in \nu(S, T)$,

$$\mathbb{E} \left[\left(\int_S^T f(t, \omega) dB_t \right)^2 \right] = \mathbb{E} \left[\int_S^T f^2(t, \omega) dt \right] \quad (5)$$

Now we are ready to define stochastic differential equations. Stochastic differential equations are defined in terms of integral rather than derivatives. This is because the most common integrator, Brownian motion, is nowhere differentiable [6]. Hence its derivative is not defined.

Definition 2.8 (Stochastic differential equation)

A process X_t satisfies the stochastic differential equation

$$dX_t = b(t, X_t)dt + \sigma(t, X_t)dB_t \quad (6)$$

if it satisfies the integral equation

$$X_t = X_0 + \int_0^t b(s, X_s)ds + \int_0^t \sigma(s, X_s)dB_s \quad (7)$$

where B_t is the standard d -dimensional Brownian motion.

In order for an SDE to possess a unique strong solution, we must impose conditions on σ and b . The proof of the following theorem is similar to the ODE case, where the proof of existence uses Picard iteration, and the proof of uniqueness uses the Itô isometry and the Lipschitz condition. The interested reader is referred to [10] or [6].

Theorem 2.1 (Existence and uniqueness of SDEs)

Let $T > 0$ and $b : [0, T] \times \mathbb{R}^d \rightarrow \mathbb{R}^d, \sigma : [0, T] \times \mathbb{R}^d \rightarrow \mathbb{R}^{d \times k}$ be measurable functions satisfying

$$|b(t, x)| + |\sigma(t, x)| \leq C(1 + |x|) \quad x \in \mathbb{R}^d, t \in [0, T] \quad (8)$$

for some constant C , $|\sigma|^2 = \sum |\sigma_{ij}|^2$, and such that

$$|b(t, x) - b(t, y)| + |\sigma(t, x) - \sigma(t, y)| \leq D|x - y| \quad x, y \in \mathbb{R}^d, t \in [0, T] \quad (9)$$

for some constant D . Let Z be a random variable which is independent of the σ -algebra generated by the Brownian motion, and such that

$$\mathbb{E}[|Z|^2] < \infty. \quad (10)$$

Then the stochastic differential equation

$$dX_t = b(t, X_t)dt + \sigma(t, X_t)dB_t, \quad 0 \leq t \leq T, \quad X_0 = Z \quad (11)$$

has unique continuous (in t) solution X_t with the property that X_t is adapted to the filtration $\mathcal{F}_t^Z$ generated by Z and $B_s, s \leq t$ and

$$\mathbb{E} \left[\int_0^T |X_t|^2 dt \right] < \infty. \quad (12)$$

Now we are ready to state the central definition for our application, namely that of the Itô diffusion.

Definition 2.9 (Itô diffusion)

An Itô diffusion is a stochastic process $X_t(\omega) = X(t, \omega) : [0, \infty) \times \Omega \rightarrow \mathbb{R}^d$ satisfying a stochastic differential equation of the form

$$dX_t = b(t, X_t)dt + \sigma(t, X_t)dB_t, \quad t \geq s; X_s = x$$

where B_t is k -dimensional Brownian motion and $b : [0, \infty) \times \mathbb{R}^d \rightarrow \mathbb{R}^d, \sigma : [0, \infty) \times \mathbb{R}^d \rightarrow \mathbb{R}^{d \times k}$ satisfy Eq. 9. We call b the drift coefficient and σ the diffusion coefficient.

Note that the assumptions of Thm. 2.1 simplify in the case of diffusion, in particular, linear growth follows from the Lipschitz condition. A very powerful result about Itô processes which we shall make use of is Itô's lemma. This is the stochastic equivalent of the chain rule in deterministic calculus.

Theorem 2.2 (Itô's lemma)

Let

$$dX(t) = udt + vdB(t)$$

be a d -dimensional Itô process. Let $f(t, x) = (f_1(t, x), \dots, f_k(t, x))$ be a C^2 map from $[0, \infty) \times \mathbb{R}^d$ into $\mathbb{R}^k$. Specifically, let f_k be continuously differentiable once in t and twice in x . Then the process

$$Y(t, \omega) = f(t, X(t))$$

is again an Itô process, whose k -th component Y_k , is given by

$$dY_k = \frac{\partial f_k}{\partial t}(t, X)dt + \sum_i \frac{\partial f_k}{\partial x_i}(t, X)dX_i + \frac{1}{2} \sum_{i,j} \frac{\partial^2 f_k}{\partial x_i \partial x_j} dX_i dX_j$$

where $dB_i dB_j = \delta_{i,j} dt$, $dB_i dt = dt dB_i = dt dt = 0$.

Note that the last line of the definition is a mnemonic. For a proof of Itô's lemma, see e.g. [10]. Another useful tool for us is the generator of the Itô diffusion, which encodes a lot of useful information about the diffusion process. Our algorithm learns the generator rather than learning the process directly.

Definition 2.10 (Generator)

Let $\{X_t\}$ be an Itô diffusion in $\mathbb{R}^d$. The generator $\mathcal{L}_t$ of X_t at time t is defined by

$$\mathcal{L}_t f(x) = \lim_{h \downarrow 0} \frac{\mathbb{E}[f(X_{t+h}) | X_t = x] - f(x)}{h}; \quad x \in \mathbb{R}^d$$

for the set $D(\mathcal{L}_t)$ of functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ where the limit exists uniformly.

The generator exists for a wider class of processes than just Itô diffusion. However, in the case of diffusion, we can relate the generator to the drift and diffusion coefficients in the following way.

Theorem 2.3 (Generator of diffusion)

The generator of the diffusion process is given by

$$\mathcal{L}_t f = b_t \cdot \nabla f + \frac{1}{2} \text{tr}(\sigma_t \sigma_t^T \nabla^2 f)$$

which in the case where σ is a scalar constant simplifies to

$$\mathcal{L}_t f = b_t \cdot \nabla f + \frac{1}{2} \sigma^2 \Delta f$$

Here f is any C_0^2 function.

We refer the interested reader to [6] for the proof. Next, we relate the generator to a useful martingale. We call this Dynkin's formula, although there are stronger versions of it with the same name [10].

Theorem 2.4 (Dynkin)

Let X_t be a diffusion process. Then M_t^f is a martingale where

$$M_t^f = f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s f(X_s) ds \quad (13)$$

so that $\mathbb{E}[M_t^f] = 0$. Here $f \in C_0^2$ is an arbitrary test function. $\mathcal{L}_t$ is the generator of the diffusion process.

Proof. The Itô formula gives

$$df(X_t) = \left(\sum_i b_i \frac{\partial f}{\partial x_i} + \frac{1}{2} \sum_{i,j} (\sigma \sigma^T)_{i,j} \frac{\partial^2 f}{\partial x_i \partial x_j} \right) dt + dM_t$$

where M_t is some martingale term. We can rewrite this as

$$M_t = f(X_t) - f(x) - \int_0^t \mathcal{L}_s f(X_s) ds.$$

□

2.2 Generative Models

A generative model allows us to interpolate between p_{simple} and p_{data} . As the name suggests, we choose p_{simple} to be some known distribution, so that we can easily generate new samples of p_{data} by sampling $z \sim p_{\text{simple}}$ and applying the model. In this thesis, we define generative modelling as follows.

Definition 2.11 (Generative modelling)

Generative modelling is the process of training a model to transform a sample from a simple distribution p_{simple} to a data distribution p_{data} .

In general, the distribution p_{data} will be unknown to us, since otherwise there would not be much to learn. Instead, we assume access to a finite number of examples of p_{data} , sampled independently, which serves to represent the true distribution. We call this our dataset.

Definition 2.12 (Dataset)

A dataset is a finite number of i.i.d. samples $z_1, \dots, z_N \sim p_{\text{data}}$.

Our dataset might be a collection of images, such as MNIST, or a collection of videos. Our aim is then to train a model to generate more images of hand-written integers or videos. We shall also need the definition of a probability path.

Definition 2.13 (Probability path)

A probability path is a time-dependent probability $\{p_t\}_{0 \leq t \leq 1}$.

An increasingly common class of generative models used are flow models. Here, one learns a vector field b_t^θ , which parametrises the ODE that carries values from p_{simple} to p_{data} [9]. Having learned b_t^θ , one can then interpolate between the distributions simply using an ODE solver.

Definition 2.14 (Flow model)

A flow model is given by:

$$\begin{aligned} \frac{d}{dt}X_t &= b_t^\theta(X_t) \\ X_0 &\sim p_{\text{simple}} \end{aligned}$$

where $b_t^\theta : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a vector field parametrised by a neural network with parameters θ .

The way flow models are trained is called flow matching. Let $\mathcal{T} \in U([0, 1])$ be a random time and $X_t \sim p_t$ our process. We construct a target vector field $u_t^{\text{target}}(x)$ and minimise the loss

$$L_{FM}(\theta) = \mathbb{E}[\|u_{\mathcal{T}}^\theta(X_{\mathcal{T}}) - u_{\mathcal{T}}^{\text{target}}(X_{\mathcal{T}})\|^2]$$

through stochastic gradient descent. One can construct the target as

$$u_t^{target}(x) = \int u_t^{target}(x|z) \frac{p_t(x|z)p_{data}(z)}{p_t(x)} dz$$

and then it holds that

$$X_0 \sim p_{simple}, \quad \frac{d}{dt}X_t = u_t^{target}(X_t) \implies X_t \sim p_t \quad (0 \leq t \leq 1)$$

[9, 4]. Unfortunately this integral is intractable, but one can show that the conditional flow matching loss, where $Z \sim p_{data}$

$$L_{CFM}(\theta) = \mathbb{E}[\|u_{\mathcal{T}}^{\theta}(X_{\mathcal{T}}) - u_{\mathcal{T}}^{target}(X_{\mathcal{T}}|Z)\|^2]$$

is the same as $L_{FM}(\theta)$, up to a constant, and so minimising one with gradient descent is equivalent to minimising the other [9], since their gradients coincide.

Flow models were preceded by (denoising) diffusion models, which, in addition to the vector field, also have a stochastic component. The aim of classic score matching, is to learn the score, from which the parametrised drift b_t^{θ} can be derived. The diffusion coefficient σ is chosen by us. Having learned the parametrised drift, we can generate new samples of our data distribution p_{data} , using one of many numerical SDE solvers.

Definition 2.15 (Diffusion model)

A diffusion model is given by:

$$\begin{aligned} dX_t &= b_t^{\theta}(X_t)dt + \sigma_t dB_t \\ X_0 &\sim p_{simple} \end{aligned}$$

where $b_t^{\theta} : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ is the parametrised drift, $\sigma_t : [0, 1] \rightarrow \mathbb{R}$ is our diffusion coefficient, and p_{simple} is our initial data distribution.

The most common way to train diffusion models is through score matching [13]. The presentation of score matching given here follows [4, 9] and is different, but equivalent to, the presentation given by [13]. They prove the following theorem

Theorem 2.5

Define the conditional and marginal vector fields $u_t^{target}(x|z)$ and $u_t^{target}(x)$

as above. Then, for diffusion coefficient $\sigma \geq 0$, we may construct an SDE which follows the same probability path, i.e.

$$X_0 \sim p_{\text{simple}}, \quad dX_t = \left(u_t^{\text{target}}(X_t) + \frac{\sigma^2}{2} \nabla \log p_t(X_t) \right) dt + \sigma dB_t \quad (14)$$

$$\implies X_t \sim p_t \quad (0 \leq t \leq 1) \quad (15)$$

In particular, $X_1 \sim p_{\text{data}}$, and this holds if we replace the marginal probability path and vector field with their conditional versions.

Now $\nabla \log p_t(x)$ is called the score, and we can write it as

$$\nabla \log p_t(x) = \int \nabla \log p_t(x|z) \frac{p_t(x|z)p_{\text{data}}(z)}{p_t(x)} dz.$$

Similarly to flow matching, we approximate the score by a neural net s_t^θ and we can derive the losses

$$L_{SM} = \mathbb{E}[\|s_{\mathcal{T}}^\theta(X_{\mathcal{T}}) - \nabla \log p_{\mathcal{T}}(X_{\mathcal{T}})\|^2]$$

and

$$L_{CSM} = \mathbb{E}[\|s_{\mathcal{T}}^\theta(X_{\mathcal{T}}) - \nabla \log p_{\mathcal{T}}(X_{\mathcal{T}}|Z)\|^2].$$

Once again, L_{SM} is intractable, but one can show that its gradient coincides with L_{CSM} .

Additionally, for Gaussian probability paths we can recover the vector field from the score or vice versa. This is why the traditional denoising diffusion models can be trained by only learning the score, since they are constructed based on Gaussian conditional probability paths [13].

Our algorithm, martingale matching, also learns the diffusion model, but using a different methodology than score matching. Instead of learning the vector field or score directly, we learn the generator of the process by enforcing a known martingale.

3 Algorithm

3.1 Training

Instead of constructing a loss using the marginal vector field, as in flow matching, or using the score, as in score matching, the idea of martingale

matching is to enforce the martingale given by Thm. 2.4, and thereby to learn the generator of the process. As we saw previously, knowledge of the generator combined with knowledge of the diffusion coefficient is enough to know the drift.

Now, we are able to construct a loss function and learn the generator by enforcing that M_t^f be a martingale. Since we already know f and σ , learning the generator means we also learn the drift term b_t^θ , as desired.

In practice, what we use is that we should have $\mathbb{E}[M_t^f] = 0$, and use deviations from this as our loss function. Thus, enforcing zero-mean allows us to learn the drift. This is how we construct our loss function.

Let

$$0 = t_0 < t_1 < \dots < t_N = T$$

be a time discretization of $[0, T]$. Let i be the index of the batch, and let $f \in \mathcal{F}$ be a function in our set of test functions $\mathcal{F}$. Then for each sample of p_{simple} and each time step, we get residual

$$\tilde{R}_{t_k}^{f,i} = f(X_{t_{k+1}}^i) - f(X_{t_k}^i) - \int_{t_k}^{t_{k+1}} \mathcal{L}_s^\theta f(X_s^i) ds$$

We use quadrature $\int_{t_k}^{t_{k+1}} \mathcal{L}_s^\theta f(X_s^i) ds \approx \Delta t_k \mathcal{L}_{t_k}^\theta f(X_{t_k}^i)$, with $\Delta t = t_{k+1} - t_k$ small, to discretise the residuals and enable us to compute them efficiently. This gives discrete residual

$$\tilde{R}_{t_k}^{f,i} = f(X_{t_{k+1}}^i) - f(X_{t_k}^i) - \Delta t \mathcal{L}_{t_k}^\theta f(X_{t_k}^i)$$

Our loss function is then simply the mean squared of these residuals. Now the loss function Eq. 16 arises naturally, since $\frac{1}{N} \sum_{i=1}^N \tilde{R} \approx \mathbb{E}[\tilde{R}]$ for large N , which we know should be 0.

$$L_1 = \frac{1}{T|\mathcal{F}|} \sum_{k=1}^T \sum_{f \in \mathcal{F}} \left(\frac{1}{N} \sum_{i=1}^N \tilde{R}^{f,i} \right)^2. \quad (16)$$

Although Eq. 16 is the mathematically cleanest version of our loss function, we made improvements not so much motivated by the theory but by their numerical superiority and displayed empirical improvements.

First, we used a Monte-Carlo approach, as in Eq. 17, which significantly speeds up training by each summand being conditioned on a different value z of our dataset.

$$L_2 = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \left(\frac{1}{N} \sum_{i=1}^N \tilde{R}^{f,i} \right)^2. \quad (17)$$

Second, we changed our loss function so that instead of being genuinely pathwise, it becomes pathwise in a weak sense. Explicitly, we used the residual

$$R_{t_k}^{f,i} = f(Y_{t_{k+1}}^i) - f(X_{t_k}^i) - \Delta t \mathcal{L}_t^\theta f(X_s^i) \quad (18)$$

where $Y_{t_{k+1}} \sim p_{t_{k+1}}(\cdot|z)$ and $X_{t_k} \sim p_{t_k}(\cdot|z)$. Note that this significantly weakens the residual, in the sense that it is now not a pathwise statement about a diffusion process X_t , but a statement about the random variables $Y_{t_{k+1}}$ and X_{t_k} . Nevertheless, it is numerically preferable, since we have that $Y_{t_{k+1}} \perp\!\!\!\perp X_{t_k} \mid Z$, where $Z \sim p_{data}$. The residual $R_{t_k}^{f,i}$ gives the loss function

$$L_3 = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \left(\frac{1}{N} \sum_{i=1}^N R^{f,i} \right)^2. \quad (19)$$

Intuitively, minimising the loss function L_3 implies weak generator learning. We formalise this as follows.

Theorem 3.1 (Weak moment matching)

Let $\tilde{X}_t \sim p_t(\cdot|z)$ be a process. If for each $f \in \mathcal{F}$ we have that $\mathbb{E}[R_t^f] = 0$, $\mathbb{E}[\mathcal{L}_s f(\tilde{X}_s)] \in C([t, t + \Delta t])$ then

$$\mathbb{E}[\mathcal{L}_t f(\tilde{X}_t)] = \mathbb{E}[\mathcal{L}_t^\theta f(\tilde{X}_t)] \quad \text{as } \Delta t \rightarrow 0. \quad (20)$$

Proof. We have that

$$\mathbb{E}[R_t^f] = \mathbb{E}[f(Y_{t+\Delta t})] - \mathbb{E}[f(X_t)] - \Delta t \mathbb{E}[\mathcal{L}_t^\theta f(X_t)] = 0. \quad (21)$$

where $X_t \sim p_t(\cdot|z)$ and $Y_{t+\Delta t} \sim p_{t+\Delta t}(\cdot|z)$ are only defined at points t and $t + \Delta t$ respectively. Using that $X_t \sim \text{Law}(\tilde{X}_t)$ and $Y_{t+\Delta t} \sim \text{Law}(\tilde{X}_{t+\Delta t})$, we can rewrite (21) as (22)

$$\mathbb{E}[\mathcal{L}_t^\theta f(\tilde{X}_t)] = \frac{\mathbb{E}[f(\tilde{X}_{t+\Delta t})] - \mathbb{E}[f(\tilde{X}_t)]}{\Delta t}. \quad (22)$$

Additionally, Dynkin gives

$$\frac{\mathbb{E}[f(\tilde{X}_{t+\Delta t})] - \mathbb{E}[f(\tilde{X}_t)]}{\Delta t} = \frac{1}{\Delta t} \int_t^{t+\Delta t} \mathbb{E}[\mathcal{L}_s f(\tilde{X}_s)] ds. \quad (23)$$

Combining (22) and (23) we get

$$\mathbb{E}[\mathcal{L}_t^\theta f(\tilde{X}_t)] = \frac{1}{\Delta t} \int_t^{t+\Delta t} \mathbb{E}[\mathcal{L}_s f(\tilde{X}_s)] ds. \quad (24)$$

Taking limits gives

$$\lim_{\Delta t \rightarrow 0} \mathbb{E}[\mathcal{L}_t^\theta f(\tilde{X}_t)] = \mathbb{E}[\mathcal{L}_t f(\tilde{X}_t)]. \quad (25)$$

□

If we now write $\epsilon = \mathbb{E}[\mathcal{L}_t f(X_t) - \mathcal{L}_t^\theta f(X_t)]$ as our error, then heuristically

$$\min_{\theta} \left(\frac{1}{N} \sum_{i=1}^N R_{\theta}^{f,i} \right)^2 \approx \Delta t^2 \epsilon^2 \quad \text{as } \Delta t \rightarrow 0. \quad (26)$$

The loss L_3 is directly justified by the weak martingale condition. In practice, we use the stronger surrogate L_{ME} , obtained by moving the square inside the empirical average. Since

$$\mathbb{E}[(R_{\theta}^f)^2] = (\mathbb{E}[R_{\theta}^f])^2 + \text{Var}(R_{\theta}^f),$$

this surrogate can be interpreted as augmenting the moment-matching objective with a variance penalty on the residuals. Although this modification is not implied by the martingale identity alone, and may in principle change the population minimiser, we find empirically that it gives more stable and faster training.

$$L_{ME} = \frac{1}{|\mathcal{F}|N} \sum_{f \in \mathcal{F}} \sum_{i=1}^N (R^{f,i})^2. \quad (27)$$

With our loss constructed, we can now create our martingale matching algorithm, which is similar in spirit to other algorithms for generative modelling, such as flow and score matching. The algorithm is given in Alg. 1. Note that we use constant scalar σ for simplicity.

Algorithm 1 Martingale Matching

Require: Neural net b_t^θ , diffusion coefficient σ , time-step Δt , dataset of samples $z \sim p_{data}$, conditional probability path $p_t(\cdot|z)$
for each minibatch of data **do**
 Sample z from minibatch.
 Sample $t \sim U[0, 1 - \Delta t]$.
 Set $x_t \sim p_t(\cdot|z)$, $x_{t+\Delta t} \sim p_{t+\Delta t}(\cdot|z)$.
 Compute loss L_{ME} .
 Update the model parameter θ via gradient descent on L_{ME} .
end for

One of the main hyperparameters thus becomes our test function, and the choice of test functions is the major design decision in any implementation of this algorithm. We want to choose a subset of functions that are sufficiently rich to learn the drift, but computationally inexpensive. Because this choice is rather arbitrary, many modern generative methods try to avoid it altogether [4]. One set of test functions is that of random Fourier features. These have the advantage of being computationally fast, particularly for higher dimensional data, as the number of functions required does not necessarily depend on the dimension of the data.

Definition 3.1 (Random Fourier features)

A random Fourier feature is a function of the type

$$f_{\omega_1}(x) = \cos(w^\top x) \quad f_{\omega_2}(x) = \sin(w^\top x)$$

where $x \in \mathbb{R}^d$ and $\omega \sim N(0, I_d)$.

Another intuitive test function to try is that of polynomials. We found empirically that these work better than the Fourier features, but they have the disadvantage that they are computationally heavier, since the number of test functions of degree k is $\binom{k+d}{k}$, where d is the dimension of our data. This means that we are restricted to both low-degree polynomials and low-dimensional data.

Definition 3.2 (Polynomial test functions)

Suppose our dataset has dimension d . Then we call the set of monomials in d variables of degree up to k the set of polynomial test functions.

Another set of test functions is Hermite polynomials. We did not find much empirical improvement over regular polynomial test functions, and hence our analysis focused on comparing random Fourier features with polynomials.

Definition 3.3 (Hermite polynomial test functions)

$$He_n(x) = (-1)^n e^{\frac{x^2}{2}} \frac{d^n}{dx^n} e^{-\frac{x^2}{2}}$$

The polynomial test functions, in addition to performing quite well empirically, have the additional advantage of having quite a nice mathematical interpretation. Consider X_t, Y_t modelled by Alg. 1, and the Markov operator defined by

$$K_{t,\Delta t}f(x) = \mathbb{E}[f(Y_{t+\Delta t})|X_t = x]. \quad (28)$$

We can use the Markov operator to characterize the minimiser of L_{ME} , as in the following theorem.

Theorem 3.2

Suppose $f \in C_0^2$. Then the minimiser of

$$\min_{F \in L^2(\Omega)} \mathbb{E}[(f(Y_{t+\Delta t}) - f(X_t) - \Delta t F)^2] \quad (29)$$

is given by

$$F = \frac{K_{t,\Delta t}f(x) - f(x)}{\Delta t} \quad p_t - a.e. \quad (30)$$

Proof. Note that we have that $X_t \sim p_t(\cdot|z)$, $Y_{t+\Delta t} \sim p_t(\cdot|z)$ and $X_t \perp\!\!\!\perp Y_{t+\Delta t} \mid Z$, so that we can condition on $X_t = x$ to give us

$$\mathbb{E}[(f(Y_{t+\Delta t}) - f(x) - \Delta t F)^2 | X_t = x]. \quad (31)$$

This is a standard least-squares problem which is minimized when by

$$\begin{aligned} F &= \mathbb{E}[f(Y_{t+\Delta t}) - f(x) | X_t = x] \\ &= \mathbb{E}[f(Y_{t+\Delta t}) | X_t = x] - f(x). \end{aligned}$$

□

Note that although F in general is not parametrisable by a fixed neural network, we learn an approximate F using $\mathcal{L}_t^\theta f(x)$. Now we may look at what the generator becomes for various test functions, by simply plugging them into the formula Thm. 2.3. For coordinate test functions $f_j(x) = x_j$ we find that

$$\mathcal{L}_t^\theta f_j = b_j^\theta \quad (32)$$

so that we simply learn the drift. However, we found better empirical performance when including higher-order terms. Second degree polynomial terms $f(x) = x_i x_j$ give us

$$\mathcal{L}_t(x_i x_j) = x_j b_i(x, t) + x_i b_j(x, t) + a_{i,j}(x, t) \quad (33)$$

where $a = \sigma \sigma^T$. In particular, for $a = \sigma^2 I$, which we use, we get

$$\begin{aligned} \mathcal{L}_t(x_i^2) &= 2x_i b_i(x, t) + \sigma^2 \\ \mathcal{L}_t(x_i x_j) &= x_j b_i(x, t) + x_i b_j(x, t). \end{aligned}$$

A key step in Alg. 1 is that of sampling $p_t(\cdot|z)$ to generate our values x . In order to do this we need to develop our concept of conditional probability paths. The most common probability path used in generative flow models is a Gaussian probability path.

Definition 3.4 (Gaussian probability path)

Let $\alpha_t = \alpha(t) \in C^1([0, 1])$ and $\beta_t = \beta(t) \in C^1([0, 1])$ be noise schedulers, that is, monotonic functions with $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$. We then define the Gaussian conditional probability path to be

$$p_t(\cdot|z) = N(\alpha_t z, \beta_t^2 I_d).$$

Then this probability path fulfills the boundary conditions

$$p_0(\cdot|z) = N(\alpha_0 z, \beta_0^2 I_d) = N(0, I_d) \quad \text{and} \quad p_1(\cdot|z) = N(\alpha_1 z, \beta_1^2 I_d) = \delta_z.$$

Score matching widely uses Gaussian probability paths. However, Gaussian probability paths have the inherent restrictions that they force p_{simple} to also be Gaussian. Hence, if one wants to interpolate between arbitrary distributions, one must use some other probability path.

However, we found that, in practice, p_{simple} is chosen for convenience provided that it is easy to sample from. Hence, Gaussian probability paths often suffice, which is why we relied on them.

3.2 Generation

Having learned our drift b_t^θ , we want to use it to generate new samples in p_{data} from our synthetic distribution p_{simple} . We can do this by simply solving the stochastic differential equation, and thus simulating its path. In our implementation we use the Euler-Maruyama solver, since it is very simple. Alg. 2 gives the pseudocode.

Algorithm 2 Euler-Maruyama

Require: Neural net b_t^θ , number of steps n , diffusion coefficient σ_t

Set $h = \frac{1}{n}$

Set $t = 0$

Sample $X_0 \sim p_{simple}$

for $i = 1, \dots, n$ **do**

Sample $\epsilon \sim N(0, I)$

$X_{t+h} = X_t + hb_t^\theta(X_t) + \sigma_t\sqrt{h}\epsilon$

Update $t \leftarrow t + h$

end for

return X_1

The Euler-Maruyama method is essentially just the Euler method for deterministic differential equations, but with some simulated noise. At each time-step, the Euler-Maruyama method takes a step in the direction predicted by our neural net b_t^θ , while adding some noise to approximate the stochastic component. This is the main part of the algorithm, namely updating the time step:

$$X_{t+h} = X_t + hb_t^\theta(X_t) + \sigma_t\sqrt{h}\epsilon$$

The Euler-Maruyama method converges in the following sense.

Definition 3.5

The time discrete approximation Y^δ , with maximum step size δ , converges strongly to the Itô process X with order $\gamma > 0$ at time T if there exists constant $C > 0$ and $\delta_0 > 0$ such that

$$\epsilon(\delta) = \mathbb{E} [|X_t - Y^\delta(T)|] \leq C\delta^\gamma.$$

One can show that the Euler-Maruyama method converges strongly with order 0.5, under the conditions of Thm. 2.1. [7] For more about the Euler-Maruyama method and other methods for solving SDEs, we refer the interested reader to [7]. We can now generate new samples of our p_{data} distribution, simply by sampling p_{simple} , and simulating the SDE using Euler-Maruyama to get from $X_0 \sim p_{simple}$ to $X_1 \sim p_{data}$.

4 Results

We implemented Alg. 1 using two of the test functions we suggested. Since we did not see a significant difference between the regular and Hermite polynomial test functions, we present the results for regular polynomial test functions here. For our implementation, we parametrised the drift using an MLP neural network of size 4×64 , 4 layers each of width 64. We used $\sigma = 0.5$, as well as 1000 SDE time-steps and a batch size of 10000 with 5000 epochs of training. The hyperparameters were chosen through initial empirical experimentation, while keeping computation fairly fast (at most a few minutes) on an Apple M3 Pro CPU.

We began by learning a mixture of Gaussians. This is perhaps the most natural distribution for us to learn, since we are using Gaussian probability paths and we have the Gaussian noise component of the stochastic differential equation as well. For this distribution, we got very good results with all of our test functions, hence we only present the result of Fourier test functions. An example of the flow of samples and trajectories of the learned model, compared with the ground-truth Gaussian probability paths, is given in Fig. 1. The ground-truth probability paths are simply Gaussian probability paths interpolating between p_{simple} which is Gaussian, and p_{data} .

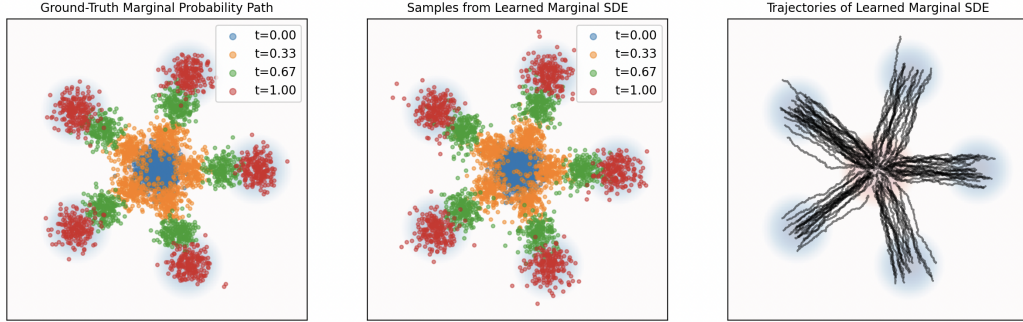


Figure 1: Illustration of learning Gaussian mixtures. (Left) Samples from the ground-truth probability path at different times; (Middle) Samples from the learned marginal SDE; (Right) Trajectories of the learned marginal SDE.

To distinguish between the quality of different test functions, we had to train on more difficult data distributions. However, we only used $2d$ distributions. This is largely because they are easy to visualize. Moreover, we encountered some problems with higher-dimensional data, which are discussed below. Fig. 2 shows the flow of data learned using Fourier test functions, compared to the ground-truth probability paths. Fig. 3 shows the results for learning a checkerboard data distribution and Fig. 4 the results for a circles distribution.

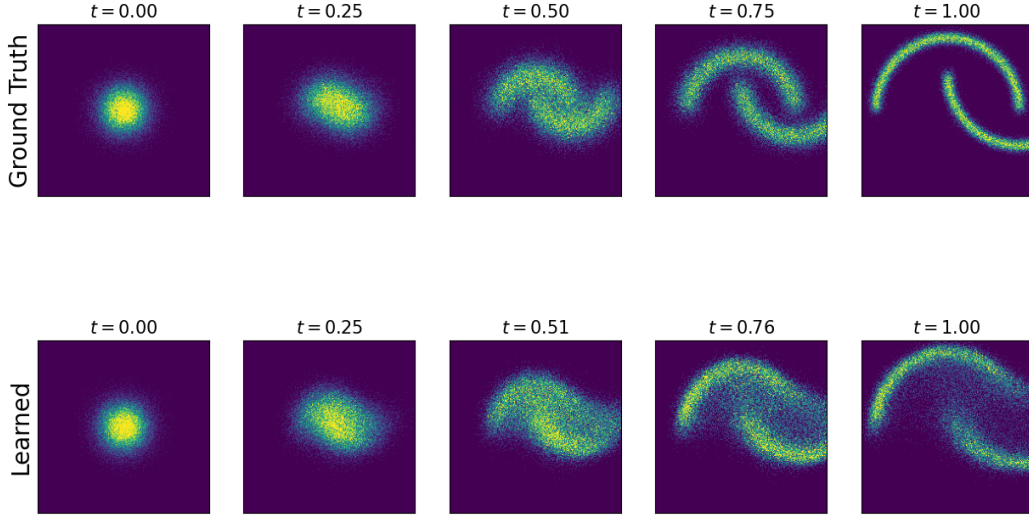


Figure 2: Illustration of learning the two moons dataset using Fourier test functions. (Top) The ground-truth Gaussian probability path, sampled at various times; (Bottom) The learned probability paths, sampled at the same times.

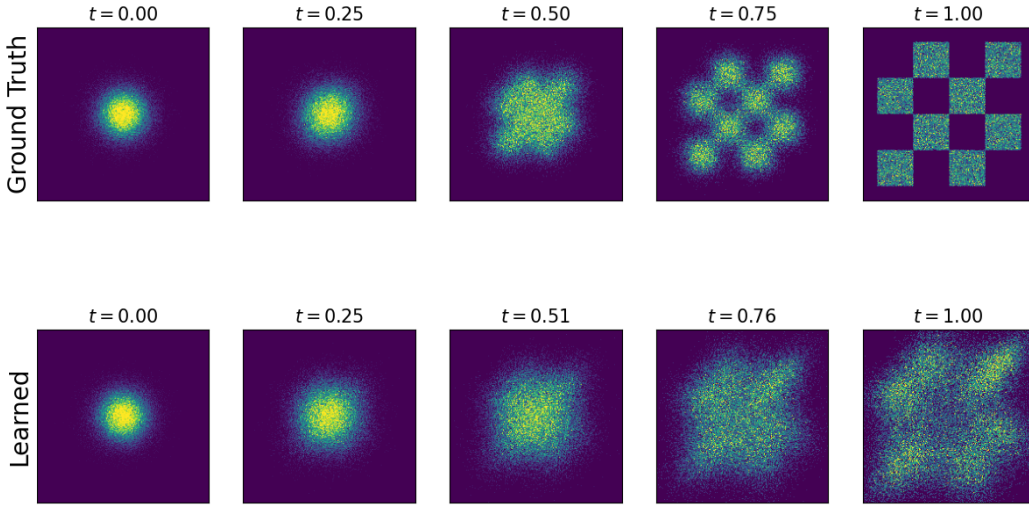


Figure 3: Illustration of learning the checkerboard dataset using Fourier test functions. (Top) The ground-truth Gaussian probability path, sampled at various times; (Bottom) The learned probability paths, sampled at the same times.

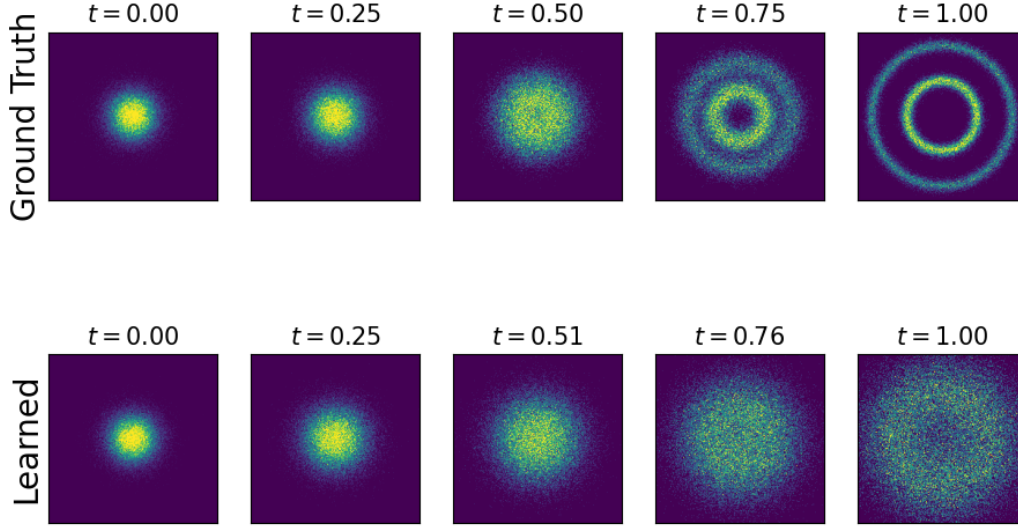


Figure 4: Illustration of learning the circles dataset using Fourier test functions. (Top) The ground-truth Gaussian probability path, sampled at various times; (Bottom) The learned probability paths, sampled at the same times.

As seen in Fig. 2 and Fig. 3, the Fourier functions seem to struggle with even slightly more complicated data than Gaussian mixtures. As soon as Fourier test functions are used to learn a drift which is clearly non-linear, the algorithm struggles. The Fourier test functions seem to especially struggle on the checkerboard, but here no stochastic process with non-zero diffusion at time $t = 1$ will learn the distribution perfectly, since the stochastic component will always make the corners slightly fuzzy. However, the Fourier test functions seem to learn very little in Fig. 4, which shows that the problem does not lie with "sharp corners".

Hence, we then implemented polynomial test functions up to the second degree on the same data sets. Fig. 5 shows two moons and Fig. 6 shows the checkerboard, both learned using polynomial test functions. We find that the polynomial test functions do much better than the Fourier features. They are likely able to better learn higher-order terms of the drift. The polynomial test functions learn the two moons dataset almost perfectly, and the checkerboard with much less fuzziness than the Fourier test functions. Moreover, Fig. 7 shows that the polynomial test functions learn the circles dataset almost perfectly.

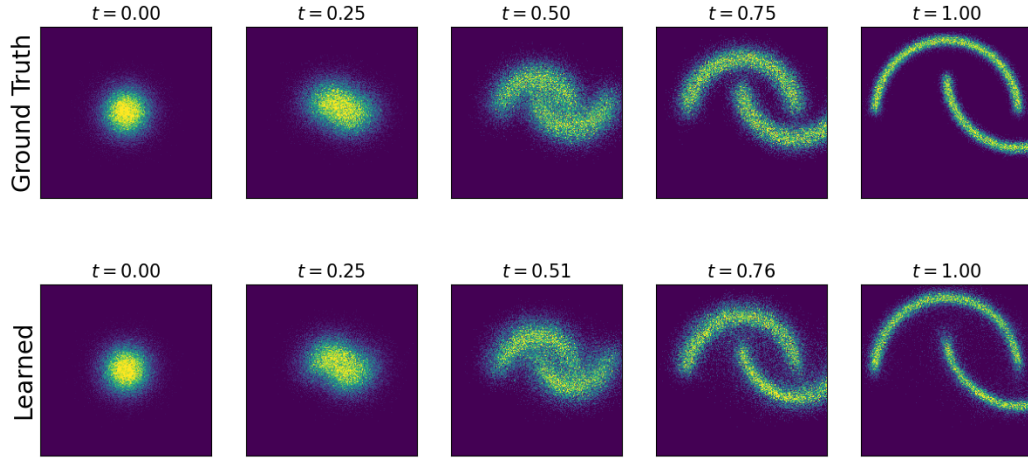


Figure 5: Illustration of learning the two moons dataset using polynomial test functions. (Top) The ground-truth Gaussian probability path, sampled at various times; (Bottom) The learned probability paths, sampled at the same times.

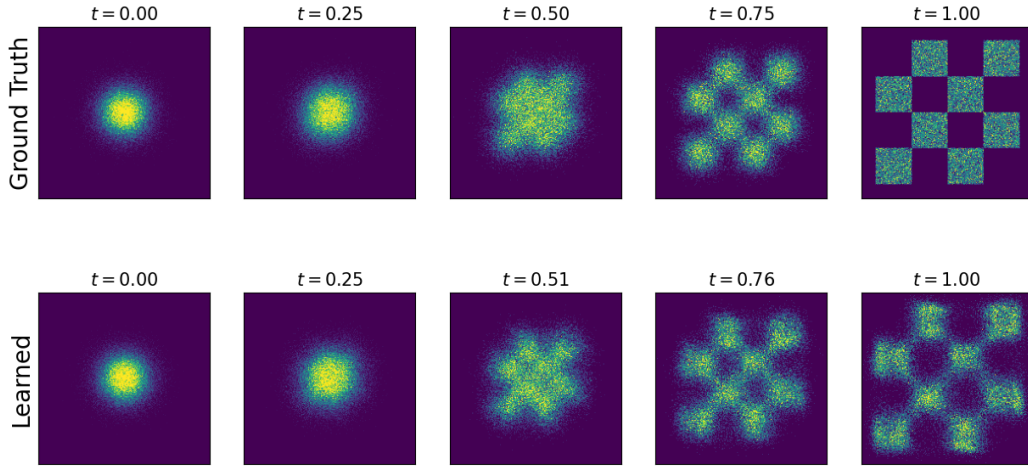


Figure 6: Illustration of learning the checkerboard dataset using polynomial test functions. (Top) The ground-truth Gaussian probability path, sampled at various times; (Bottom) The learned probability paths, sampled at the same times.

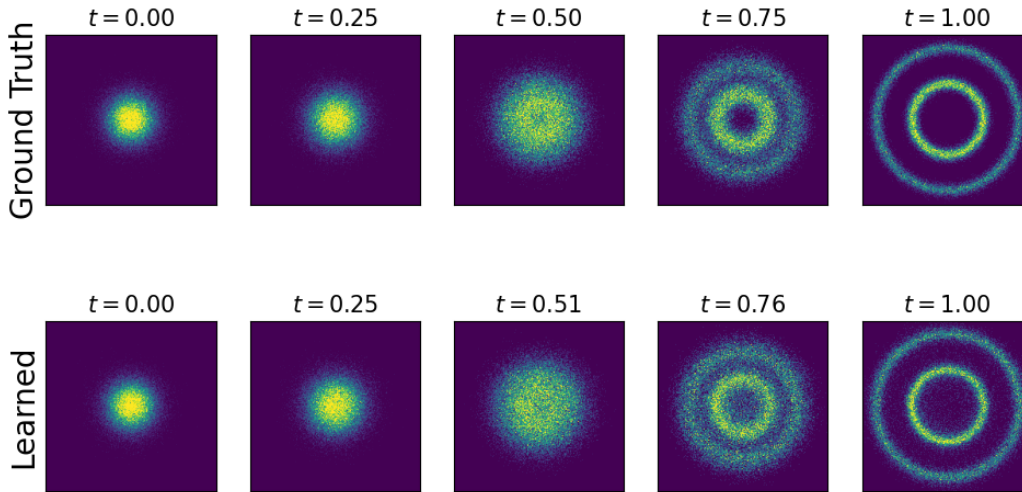


Figure 7: Illustration of learning the circles dataset using polynomial test functions. (Top) The ground-truth Gaussian probability path, sampled at various times; (Bottom) The learned probability paths, sampled at the same times.

We also attempted to learn higher-dimensional data, such as the MNIST dataset of handwritten digits with 28×28 pixels, which has dimension $28 \times 28 = 784$. However, we encountered the problem that our Fourier test functions were not good enough, whereas the number of our second-degree polynomial test functions grow quadratically. This means that in the MNIST case, we have $\binom{784+2}{2} = 308505$ test functions. This significantly slows down our computation, and more importantly, requires too much memory to run on a regular computer.

5 Conclusion

The results we got applying martingale matching to toy data were very encouraging. Clearly we are able to learn something useful about our data simply by enforcing the martingale given in Eq. 13. The main architectural difficulty we encountered was finding test functions which give us good results while being computationally cheap.

Hence, there are a multitude of ways to extend this work, since the primary design choice for martingale matching is how we choose test functions.

One option would be to look at different test functions than Fourier features or polynomials. Our choice of test functions was somewhat arbitrary, and there are surely alternatives that would perform better. Another would be to try to mitigate the weaknesses of different test functions by combining them in various ways, to better represent the whole space of test functions.

Perhaps the most promising extension would be to learn the test functions. If one were to parametrise them by their own neural nets, then one could simply train them while training the drift. This would remove some of the design choices that have to be made while implementing the martingale matching algorithm. More importantly, it would likely decrease the number of test functions needed and significantly improve the scalability to higher dimensions. This is because a dataset like MNIST, although it has 784 pixels, in reality has many fewer true dimensions. Hence, many of the relationships captured by mixed-term polynomials likely do not contribute meaningfully to the learning process, and hence could be discarded. We hypothesize that the machine could do quite a good job at choosing the meaningful dimensions through learning.

References

- [1] Brian D.O. Anderson. “Reverse-time diffusion equation models”. In: *Stochastic Processes and their Applications* 12.3 (May 1982), pp. 313–326. DOI: 10.1016/0304-4149(82)90051-5.
- [2] Jonathan Ho, Ajay N. Jain, and Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.
- [3] Peter Holderrieth and Ezra Erives. *An Introduction to Flow Matching and Diffusion Models*. 2025. arXiv: 2506.02070 [cs.LG]. URL: <https://arxiv.org/abs/2506.02070>.
- [4] Peter Holderrieth et al. “Generator Matching: Generative modeling with arbitrary Markov processes”. In: *International Conference on Learning Representations*. Vol. 2025. 2025, pp. 52153–52219.
- [5] Olav Kallenberg. *Foundations of Modern Probability*. Springer, Jan. 2002. ISBN: 9780387953137.
- [6] Ioannis Karatzas and Steven E Shreve. *Brownian Motion and Stochastic Calculus*. New York: Springer, 1998. ISBN: 9780387976556.

- [7] Peter E Kloeden and Eckhard Platen. *Numerical Solution of Stochastic Differential Equations*. Berlin: Springer, 1999. ISBN: 9783540540625.
- [8] Yaron Lipman et al. “Flow Matching for Generative Modeling”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [9] Yaron Lipman et al. *Flow Matching Guide and Code*. 2024. arXiv: 2412.06264 [cs.LG]. URL: <https://arxiv.org/abs/2412.06264>.
- [10] Bernt Øksendal. *Stochastic Differential Equations : An Introduction with Applications*. Heidelberg: Springer, 2013. ISBN: 9783540047582.
- [11] Jascha Sohl-Dickstein et al. “Deep Unsupervised Learning using Nonequilibrium Thermodynamics”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, July 2015, pp. 2256–2265.
- [12] Yang Song and Stefano Ermon. “Generative Modeling by Estimating Gradients of the Data Distribution”. In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019.
- [13] Yang Song et al. “Score-Based Generative Modeling through Stochastic Differential Equations”. In: *International Conference on Learning Representations (ICLR)*. 2021.